

Kwant — Theory and Practice · User Manual

Contents

1. What it is	2
2. Installation	2
2.1 Windows (one command)	2
2.2 Linux, macOS, or an existing conda	2
2.3 Why <code>numpy<2.5</code>	3
2.4 Verify	3
3. Running the notebooks	3
4. Map of the course	3
Part I — Kwant and transport theory	3
Part II — Topological matter	4
5. Exercises and solutions	5
6. The support scripts	5
6.1 <code>verify_kwant.py</code>	5
6.2 <code>test_thread_safety.py</code>	5
7. Logs and audit	5
8. Tests	5
9. Editing and re-executing	6
10. The <code>dev/</code> directory	6
11. Parallelism — read before you parallelise	6
12. Upstream findings	6
12.1 The course	6
13. Workflows	7
14. Features and limitations	7
14.1 Features	7
14.2 Known limitations	7
15. Licence and attribution	8

1. What it is

Kwant — Theory and Practice is a course on quantum transport in one executed Jupyter notebook, built against [Kwant](#) 1.5. Each of its 26 sections states a piece of theory, builds the corresponding tight-binding system with Kwant, computes what the theory predicts, and compares the two in the same cell. A companion notebook works every one of the 25 exercises with assertions.

File	Content
Kwant_Theory_and_Practice.ipynb	86 cells (53 code, 33 markdown), 68 figures, 25 exercises
Kwant_Exercises_Solutions.ipynb	51 cells (24 code, 27 markdown), 18 figures, 31 assertions
install_kwant_windows.ps1, .bat	Windows installer
verify_kwant.py	installation check with physics identities
test_thread_safety.py	regression test for the MUMPS thread crash
tests/	pytest suite
dev/	standalone scripts the notebooks were built from
course/	the undergraduate course: slides (course/deck/index.html), a PDF of the deck, handout, lecturer notes
scripts/	weekly upstream watch (watch_upstream.py) and its scheduler entry

Section 14 lists every feature and every known limitation in one place.

2. Installation

2.1 Windows (one command)

Double-click `install_kwant_windows.bat`, or from PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\install_kwant_windows.ps1 [-EnvName kwant] [-PythonVer 3.13] [
```

Parameter	Default	Meaning
-EnvName	kwant	conda environment to create; also the Jupyter kernel name
-PythonVer	3.13	Python version (conda-forge has Kwant 1.5.0 builds for 3.11–3.13)
-Force	off	remove and recreate the environment if it exists
-SkipInit	off	do not run <code>conda init</code> for PowerShell and <code>cmd.exe</code>

What it does, in order: finds conda (or downloads and silently installs Miniforge3 from GitHub); creates the environment from conda-forge with `kwant`, `numpy<2.5`, `scipy`, `matplotlib`, `ipykernel`, `jupyterlab`; installs the optional extras best-effort (`python-mumps`, `sympy`, `qsym`, `plotly`, `ipynb`) so one unavailable package cannot fail the install; registers a Jupyter kernel named `kwant` (“Python (kwant)”), which is the name both notebooks are pinned to; runs `conda init`; runs a real transport calculation to verify. It needs network access and about 2 GB of disk; it does not touch any other environment.

2.2 Linux, macOS, or an existing conda

```
conda create -n kwant -c conda-forge kwant "numpy<2.5" scipy matplotlib sympy python-mumps ipykernel jupyterlab
```

```
conda activate kwant
python -m ipykernel install --user --name kwant --display-name "Python (kwant)"
python verify_kwant.py
```

`pip install kwant` is not recommended: Kwant has compiled extensions and needs MUMPS for acceptable speed; conda-forge ships both.

2.3 Why numpy<2.5

numpy 2.5.0 (June 2026) removed the 2-vector form of `np.cross`, which the released Kwant 1.5.0 still uses in `kwant.physics.magnetic_gauge`. On numpy 2.5 that function raises `ValueError: Both input arrays must be (arrays of) 3-dimensional vectors` for every 2-D system. The fix has been on Kwant's main branch since January 2025 but is in no release. The notebooks do not call `magnetic_gauge` (they use explicit Peierls phases), so they run on any numpy 2.x; the pin only protects users who want that function. `verify_kwant.py` reports the breakage as a warning. Remove the pin once a Kwant release carries the fix.

2.4 Verify

```
python verify_kwant.py
```

runs seven groups of checks: core imports and versions; optional components (MUMPS, `kwant.continuum/sympy`, `kwant.qsymm`, `plotly`); conductance quantisation of a clean wire at four energies (T must equal the number of propagating modes to 1e-6); exact identities (S-matrix unitarity to 1e-9, the sum rule $T + R = N$, wave-function count, the density and current operators, `magnetic_gauge`); sparse diagonalisation of a closed dot; the lead band structure; and symbolic discretisation with `kwant.continuum`. It ends with a one-line result and the interpreter path to select in your editor.

3. Running the notebooks

- Open `Kwant_Theory_and_Practice.ipynb` in JupyterLab or VS Code and choose the kernel **Python (kwant)**. If your kernel has another name, select it once; Jupyter remembers the choice per notebook.
- Use **Run All** (or *Restart kernel and run all cells*). Figure numbers come from a global counter defined in cell 2, so they are only sequential under a full run; re-running a single cell advances the counter (harmless).
- Timings on a laptop with MUMPS: main notebook about 4½ minutes (280 s), solutions about 2 minutes (132 s), measured 2026-08-31 on the shipped 86- and 51-cell notebooks. Without MUMPS (SciPy's SuperLU) expect 3–5× longer.
- Cell 1 prints the versions in use and which sparse solver is active, and mutes three library warnings (see 14.2).
- The notebooks are self-contained: no data files, no downloads, no imports between them.

4. Map of the course

Part I — Kwant and transport theory

§	Title	Theory	Kwant
1	From the Schrödinger equation to a tight-binding lattice	finite differences, the tight-binding limit	—
2	The object model	graphs, sites, symmetries	<code>Builder</code> , <code>lattice</code> , <code>HoppingKind</code> , <code>finalized()</code>
3	First transport calculation — a quantum wire	conductance quantisation	<code>smatrix</code> , <code>attach_lead</code>

§	Title	Theory	Kwant
4	Scattering theory: Landauer–Büttiker and Fisher–Lee	derived, then unitarity and sum rule checked	SMatrix
5	Shapes, spatially varying values, runtime parameters	quantum-well resonances	shape , value functions, params
6	Spin	Rashba, Zeeman	norbs , matrix values
7	Band structure and closed systems	Bloch theorem, Fock–Darwin	physics.Bands , hamiltonian_submatrix
8	Beyond square lattices — graphene	Dirac cones, Klein tunnelling	lattice.honeycomb , sublattices
9	Superconductivity	BdG, Andreev reflection, discrete symmetries	particle-hole in Builder , conservation_law
10	Local observables	density, current	operator.Density , operator.Current
11	Visualisation	—	plot , plotter.map , plotter.current , 3D
12	The kernel polynomial method	Chebyshev expansion of the DOS	kpm.SpectralDensity
13	kwant.continuum	symbolic → discretised Hamiltonians	discretize , lambdify
14	Magnetic fields	Peierls substitution, Landau levels, Hofstadter butterfly	value functions with phases, discretize_landau
15	Solvers, performance and scaling	sparse direct solvers; parallel sweeps	solvers.mumps , solvers.sparse , threads vs processes
16	Pitfalls	the things that actually go wrong	—

Part II — Topological matter

§	Model	Invariant computed
17	SSH chain	winding number; edge states
18	Kitaev chain	Pfaffian Z ; Majorana zero modes
19	Majorana nanowire	topological phase diagram; zero-bias peak in transport
20	Thouless pump	Chern number in (k, t) ; pumped charge
21	Haldane model	Chern number (Fukui–Hatsugai–Suzuki); Berry curvature on the torus (3D)
22	Kane–Mele model	Z ; helical edge density
23	p+ip superconductor	chiral Majorana edge; vortex modes
24	BBH quadrupole	quadrupole moment; corner modes
25	Weyl semimetal	sliced Chern numbers; Fermi arcs
26	3D topological insulator	surface Dirac cone; layer-resolved spectrum

Section 27 is a reference shelf; the final cell states sources, attribution and the licence.

5. Exercises and solutions

Exercises are collected at the end of each part (E1.1–E1.11, E2.1–E2.14). Each carries a difficulty mark — ◦ direct, • requires thought, mini-project — and an origin flag: *from the Kwant tutorial*, *after topocondmat.org*, *after Asbóth*, or *original*. E1.11 and E2.14 are pencil-and-paper.

Kwant_Exercises_Solutions.ipynb has one section per exercise with the same tag, a worked solution, and at least one `assert` stating the expected physics (a quantised value, a decay law, a sign). The test suite checks that the two sets of tags match one to one.

6. The support scripts

6.1 `verify_kwant.py`

```
python verify_kwant.py [--outdir DIR] [--log-dir DIR] [--json] [--verbose | --quiet] [--version]
```

Option	Default	Meaning
<code>--outdir</code>	<code>.</code>	where <code>verify_kwant_summary.json</code> and the <code>logs/</code> folder go
<code>--log-dir</code>	<code><outdir>/logs</code>	where the audit log goes
<code>--json</code>	off	also print the summary JSON on stdout
<code>--verbose</code>	off	show details (default solver name, tracebacks)
<code>--quiet</code>	off	print only the one-line result

Exit code 0: all checks passed (warnings allowed); 1: at least one failure; 2: Kwant could not be imported. The summary JSON has `kwant`, `numpy`, `scipy`, `optional` (`mumps`, `continuum`, `default_solver`), `ok`, `warnings`, `failures`, `log`, `exit_code`.

6.2 `test_thread_safety.py`

```
python test_thread_safety.py [--workers 4] [--energies 64] [--tol 1e-12] [--no-canary]
                             [--canary-timeout 300] [--outdir DIR] [--log-dir DIR] [--json] [--verbose |
```

Check 1 runs an energy sweep with `kwant.solvers.sparse` in a thread pool and requires it to equal the serial sweep to `--tol`. Check 2 (the canary) runs the *crashing* pattern — `kwant.smatrix` with MUMPS in threads — in a child process and reports whether it crashed (expected) or survived (a future Kwant/python-mumps may make it safe). `--no-canary` skips check 2. Exit code 0 means check 1 passed. The summary JSON has `safe_path_maxdiff` and `canary` (`crashed`, `survived`, `skipped`, or `skipped: MUMPS not installed`).

7. Logs and audit

Every run of either script writes `<outdir>/logs/<script>_<YYYYmmdd_HHMMSS>.log` containing the exact command line, Python and platform, package versions, every message at every verbosity, and the exit code; and `<outdir>/<script>_summary.json`. `--log-dir` moves the log. Both are gitignored.

8. Tests

```
python -m pytest tests -q # ~10 s
KWANT_NB_EXECUTE=1 python -m pytest tests/test_execute_notebooks.py -q # ~6 min
```

`tests/test_notebooks.py` asserts, without a kernel: both notebooks pinned to kernel `kwant`; every code cell executed with no error output; cell, code-cell and figure counts (86/53/68 and 51/24/18); 25 exercises matched one-to-one to solution sections; no personal paths or e-mail addresses in any cell or output; the attribution cell present. `tests/test_scripts.py` asserts the CLI contract of both scripts (every option in `--help` with its default, log and JSON written, `--log-dir`, `--quiet`). `tests/test_execute_notebooks.py` executes both notebooks from a cold kernel into a temporary directory and requires zero errors and the committed figure count; it runs only with `KWANT_NB_EXECUTE=1`. CI runs all three on Linux and Windows with conda-forge Kwant + MUMPS.

9. Editing and re-executing

Change source cells, never outputs. Then re-execute in place:

```
python -m jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor.kernel_name=kwant Kwa
```

and run the tests. If a count changed, update `tests/test_notebooks.py`, `README.md`, `CITATION.cff` and section 1 of this manual together.

10. The dev/ directory

Standalone scripts from which the notebooks were assembled, kept as the runnable per-model reference: `dev18_ssh.py` ... `dev27_ti3d.py` (Part II), `dev_butterfly3d.py`, the cell sources `cells_a-c.py` and `sol_part1-2.py`, and the thread-crash reproductions (`thread_repro.py`, `thread_scipy.py`, `thread_variants.py`, `proc_test.py`, `bench.py`). `dev/build-history/` holds the four one-shot scripts that wrote or patched the notebooks; they exit unless `KWANT_NB_REBUILD=1` is set, because re-running them would duplicate cells.

11. Parallelism — read before you parallelise

`kwant.smatrix` releases the GIL, so a `ThreadPoolExecutor` looks like the obvious way to speed up an energy sweep. **With MUMPS installed this segfaults the process:** MUMPS is not re-entrant, and giving each thread its own solver instance does not help. Section 15 of the notebook demonstrates the two safe options — `kwant.solvers.sparse` (SuperLU) in threads, or one process per worker — and `test_thread_safety.py` guards the notebook against regressing. A patch that makes `kwant.solvers.mumps` thread-safe was prepared for the Kwant project (`docs/drafts/patches/0001-*.patch`).

12. Upstream findings

Studying Kwant for this course produced findings for the Kwant project, documented in `docs/01-upstream-audit.md` with statuses in `docs/02-findings-backlog.md`: the MUMPS thread crash (unreported; patch prepared); the numpy 2.5 breakage of `magnetic_gauge` in the released 1.5.0; the `site_color` callable receiving a `Site` for a builder but an index for a finalized system (documentation gap); the meaning and periodicity of `wraparound` momenta (documentation gap); `kwant.plotter` resetting all warning filters after a 3D plot (patch prepared). Issue texts and patches are in `docs/drafts/`.

12.1 The course

`course/deck/index.html` is a reveal.js deck (offline) of ten sections and 79 slides in **one linear sequence** — previous/next only, no build animations; inside each section the slides go from plain language to Kwant code to the mathematics, and a generated opener with a level-coloured agenda starts each section. **S** shows the speaker notes. All 68 figures of the main notebook appear, each under its full caption with its section and cell (`course/figures/figures.json` maps them). Fallbacks for any projector: `course/deck/slides.pdf` (landscape, one slide per page) and `course/deck/slides.html` (the same slides as one static page, no JavaScript). `course/handout/handout.pdf` is the A4 companion (reference card, key equations, the models table, pitfalls);

course/lecturer_notes.md has every slide's text and notes in order. Rebuild after changing the notebook or course/deck/content_en.py with `python course/build_course.py`; `tests/test_course.py` reads the committed files back.

13. Workflows

- **Study:** Run All, then read top to bottom; each section is self-contained except that Part II reuses two helpers: `chern_fhs` (defined in section 20) and `bloch_hamiltonian` (section 21).
- **Teach:** the exercises at the end of each part are the assignment; the solutions notebook is the marking key (its assertions are the rubric).
- **Check a new Kwant or numpy release:** `python verify_kwant.py`, then `KWANT_NB_EXECUTE=1 python -m pytest tests -q`.

14. Features and limitations

14.1 Features

- Executed, re-executable course: 86 cells, 68 figures, all from a cold run; the test suite re-executes and requires zero errors and the same figures.
- Theory derived next to the numerical check in every section of Part I.
- Ten topological models with their invariants computed from the Kwant system (winding, Pfaffian, FHS Chern, Z, sliced Chern, quadrupole).
- 25 origin- and difficulty-flagged exercises; solutions notebook with 31 assertions; tags matched one-to-one by the tests.
- ~126 literature citations with years; 10 spot-checked against Crossref.
- Windows installer with self-verification; conda one-liner elsewhere.
- `verify_kwant.py`: physics-identity installation check, CLI, log, JSON.
- `test_thread_safety.py`: MUMPS thread-crash regression test, CLI, log, JSON.
- Test suite and CI (Linux, Windows) covering notebook invariants, CLI contract and cold-kernel execution.
- Apache-2.0 licence; attribution cell for the three sources used as models; `CITATION.cff`; `AGENTS.md` for AI agents.

14.2 Known limitations

- **Pinned to Kwant 1.5** (and `numpy<2.5` for `magnetic_gauge`, see 2.3); API changes in a future Kwant release may require edits. `verify_kwant.py` is the first thing to run after upgrading.
- **Installer is Windows-only**; other platforms use the conda lines in 2.2.
- **Figure numbering is a global counter**: sequential only under Run All.
- **Three library warnings are muted** in cell 1 (kwant 1.5.0's 3D plotter calling a matplotlib function deprecated in 3.10 — fixed upstream but unreleased; mpmath's deprecated `bitcount` via sympy; kwant's `plotter.map` overflow note for delta-like densities). They are muted by wrapping `warnings.showwarning`, not with `filterwarnings`, because `kwant.plotter` resets all warning filters on every 3D plot (reported upstream). Only those three messages are muted; everything else shows.
- **Timings and eigensolver signs vary run to run**: printed wall times differ; degenerate $\pm E$ BdG pairs are printed as $|E|$.
- **Tested on one machine** by the author (Windows 10, Python 3.13, conda-forge Kwant 1.5.0 + python-mumps 0.0.6, numpy 2.5.1 — i.e. *above* the pin, with `magnetic_gauge` unusable and unused); the pinned `numpy<2.5` configuration is the one CI installs on Linux, Windows and macOS, not the author's laptop.
- **MUMPS threads**: never call `kwant.smatrix` from threads with MUMPS installed (section 11 of this manual).
- **Part II cross-cell dependency**: sections 21–26 reuse `chern_fhs` from section 20 and sections 22–26 reuse `bloch_hamiltonian` from section 21; run Part II in order.
- **No plotly/interactive figures**; 3D plots are static matplotlib.

15. Licence and attribution

Apache License 2.0 (see `LICENSE`). The Kwant tutorial (BSD-2), topocondmat.org (CC BY-SA 4.0 text, BSD-3 code) and Asbóth, Oroszlány & Pályi's *A Short Course on Topological Insulators* served as models for some exercises and for the pedagogical route through Part II; each such exercise is flagged, nothing is reproduced verbatim, and the notebook's final cell states this. Kwant is © the Kwant authors, BSD-2. Cite with `CITATION.cff`.